

FIREWATCH API (Flask) — Documentación Técnica

Estructura del código, endpoints y modelo de datos

Versión: v1.0 — Fecha: 2025-09-25

Enlace repositorio GitHub: https://github.com/Vilicaprogramer/firewatch_api_flask.git

1. Visión general

Esta documentación describe la API desarrollada con Flask para el proyecto FIREWATCH. Incluye un servicio web con endpoints para generación de gráficos (DDOS, Phishing y Login), generación de informes PDF, y un logger en segundo plano que inserta registros de prueba en la base de datos.

El despliegue está pensado para Render.com mediante Gunicorn, con dependencias definidas en requirements.txt y variables de entorno para la conexión a PostgreSQL.

2. Estructura del repositorio

La estructura mínima del proyecto es la siguiente:

- app.py: aplicación Flask con endpoints, arranque del logger y utilidades de visualización y descarga de PDF.
- limpieza_datos.py: módulo con funciones de limpieza, enriquecimiento e inserción para DDOS y Login (y un bloque de Phishing comentado).
- render.yaml: configuración de servicio para Render.com (web service, build/start commands, y versión de Python).
- requirements.txt: dependencias requeridas para ejecutar el servicio en producción.

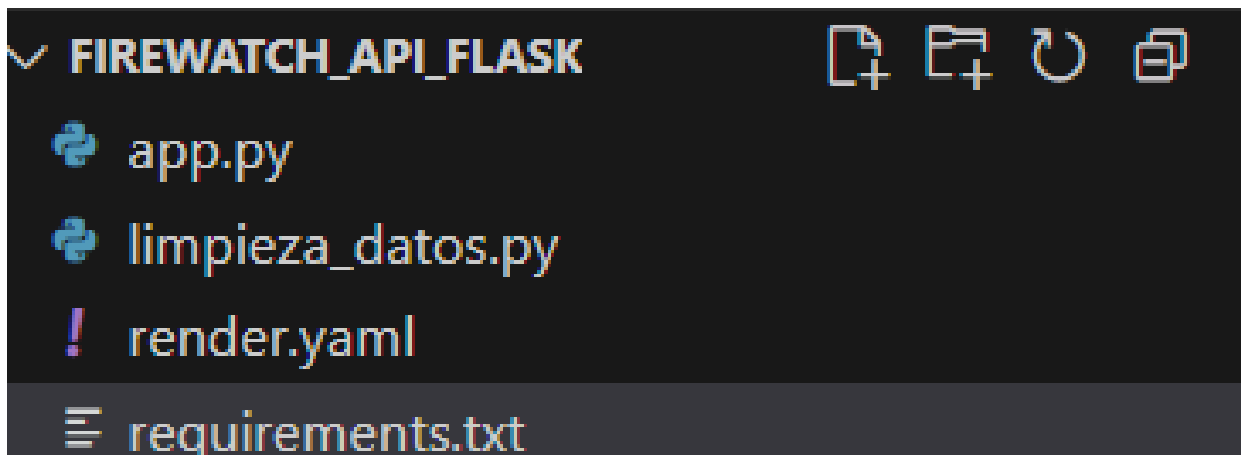


Figura 1. Estructura del proyecto.

3. app.py — Servicio Flask

Puntos clave:

- Configuración base: instancia de Flask, CORS y modo DEBUG.
- Conexión a BD: función `get_connection()` que toma las credenciales desde variables de entorno: `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT`.
- Utilidades de gráficos: `create_pie_chart(df_group, title)` genera un pastel (Plotly) en JSON para el front.
- Logger en segundo plano:
 - `background_logger()`: carga datasets de ejemplo (login y ddos) desde S3 y, cada 30s, selecciona un registro al azar para procesarlo.
 - `malware_type_detection(record)`: enruta a `clean_data_ddos(record)` si tiene 'Destination Port', de lo contrario usa `tres_en_uno(record)`.
 - Endpoints de control:

POST /start-logging → inicia el thread si no está corriendo.

POST /stop-logging → detiene el logger.

- Endpoints de gráficas (GET):
 - `/grafica_ddos`: lee de logs los indicadores ('BENIGN', 'XSS', 'Brute Force', 'Sql Injection'), agrupa y devuelve un pastel con título 'DDOS'.
 - `/grafica_phishing`: agrupa 'Correo seguro' y 'Posible phishing'.
 - `/grafica_login`: agrupa indicadores de login ('Robo de credenciales', 'Cuenta comprometida', 'Ataque fallido', 'Login válido').
- Endpoints de informes (POST):
 - `/download_pdf_phishing`: genera un PDF con los atributos enriquecidos de la URL (VirusTotal).
 - `/download_pdf_login`: genera un PDF con el detalle de login y enriquecimiento (AbuseIPDB).
 - `/download_pdf_ddos`: genera un PDF con métricas de flujo, mapeo por puerto y metadatos.
- Ejecución local: `app.run(host='0.0.0.0', port=5000, debug=True)`.

4. limpieza_datos.py — Pipelines y persistencia

4.1. DDOS — `clean_data_ddos(archive_dic)`

- Selección de columnas relevantes de tráfico (duración, paquetes, bytes/s, flags, etc.).
- Mapeo de Label → Score (Critical/High/Moderate/Benign).
- Mapeo de Label → Severity (3/2/1/0) y Score → Tipo (Incidencia/Alerta/Info).
- Normalización del campo Indicadores (elimina prefijo 'Web Attack ')?').
- Enriquecimiento por puerto (HTTP, DNS, HTTPS y puertos no estándar): columnas Estandar, Description, Ataques/CVEs tipicos, Como proteger.
- Generación de `ddos_id` (UUID reducido), Date y Time.
- Inserción en tablas:
 - logs (id, company_id, type, indicators, severity, date, time, actions_taken).
 - ddos (log_id + todas las métricas y los metadatos enriquecidos).

4.2. LOGIN — `tres_en_uno(dict)`

- Limpieza: `clean_data_login2` calcula Date/Time desde 'Login Timestamp' (forzando año 2025) y define Severity/Tipo/Indicadores según reglas lógicas sobre 'Login Successful', 'Is Attack IP' e 'Is Account Takeover'.
- Enriquecimiento: `enrich_login_record(record_dict)` consulta AbuseIPDB (`check_ip_info`) y añade `countryCode`, `abuseConfidenceScore`, etc.
- Persistencia con `insert_into_db_debug`:
 - Inserta primero en logs (RETURNING id).
 - Luego en login, referenciando `log_id` e incluyendo `login_timestamp`, `user_id`, `round_trip_time`, `ip_address`, `country`, `asn`, `user_agent` y campos de enriquecimiento.

4.3. PHISHING (comentado)

- Pipeline de scoring (`HasPopup`, `IsDomainIP`, etc.) para clasificar 'Correo seguro' vs. 'Posible phishing'.
- Llamadas a VirusTotal (`get_url_info` → `network_location`) para enriquecer con `whois`, `dns_records`, certificados y categorías.
- Inserción en logs y en tabla phishing (cuando se active).

5. Despliegue (Render.com)

- `render.yaml` define un servicio web Python (plan free).
- `buildCommand`: `'pip install --upgrade pip && pip install -r requirements.txt'`.
- `startCommand`: `'gunicorn app:app --bind 0.0.0.0:$PORT'`. (Asegurar archivo `app.py` y objeto 'app').
- Variables de entorno recomendadas:

`DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT`.

(y claves API si aplica).

- Recomendaciones: usar `psycpg2-binary` en despliegues, desactivar `DEBUG` en producción y considerar un Worker/cron para tareas continuas si el plan free hiberna.

6. Dependencias

`requirements.txt`:

Flask, flask-cors, plotly, psycpg2, pandas, seaborn, gunicorn, reportlab, requests, python-dotenv.

7. Modelo de datos (PostgreSQL)

Tablas principales y relaciones:

- `companies` (`id`, `name`) — catálogo de empresas.
- `users` (`id`, `company_id`, `username`, `email`, `password`, `role`, `logged`) → FK a `companies.id`.
- `logs` (`id`, `company_id`, `status?`, `type`, `indicators`, `severity`, `date`, `time`, `actions_taken`) → FK a `companies.id`.
- `login` (`id`, `log_id`, `login_timestamp`, `user_id`, `round_trip_time`, `ip_address`, `country`, `asn`, `user_agent`, `country_code`, `abuse_confidence_score`, `last_reported_at`, `usage_type`, `domain`, `total_reports`) → FK a `logs.id`.
- `ddos` (`id`, `log_id`, métricas de flujo..., Indicadores, Score, Severity, Tipo, Estandar, Description, Ataques/CVEs tipicos, Como proteger, date, time) → FK a `logs.id`.

- phishing (id, logs_id, url, status, malicious, suspicious, undetected, harmless, timeout, whois, tags, dns_records, last_dns_records_date, registrar, expiration_date, tld, issuer, subject_cn, cert_not_before, cert_not_after, cert_key_size, thumbprint_sha256, reputation, popularity_ranks, jar, categories) → FK a logs.id.

La figura siguiente muestra el diagrama de relaciones.

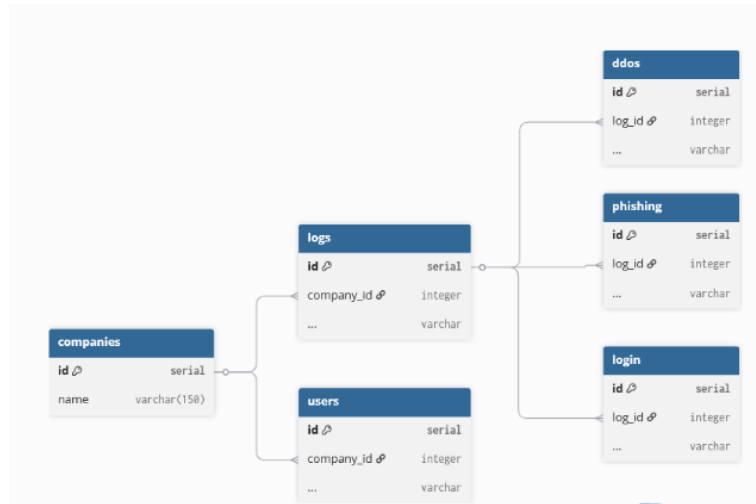


Figura 2. Diagrama de la base de datos (PostgreSQL).

8. Seguridad y buenas prácticas

- Nunca exponer claves ni cadenas de conexión en el código fuente: usar variables de entorno/secret files.
- Validar y sanear inputs en endpoints POST (p. ej., esquemas con Pydantic/Marshmallow si se amplía).
- Gestionar sesiones y CORS según dominios permitidos en producción.
- Manejo de errores y logging estructurado (app.logger) para auditoría.
- Limitar concurrencia del thread de background y considerar colas (RQ/Celery) si la carga crece.

9. Anexos

- Endpoints resumidos:

GET /grafica_ddos	POST /start-logging
GET /grafica_phishing	POST /stop-logging
GET /grafica_login	POST /download_pdf_phishing
	POST /download_pdf_login
	POST /download_pdf_ddos

- Variables de entorno mínimas: DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT.